AWS Practical Documentation

AWS IAM-Based EC2 Access Control and EBS Persistence Experiment

Practical Report

AWS Services Used

IAM — EC2 — EBS

Region: `ap-south-1`
Availability Zone: `ap-south-1b`

1. Objective

The objective of this experiment is to understand how AWS IAM controls access to EC2 resources through users, groups, and policies, and to demonstrate how EBS volumes can retain data independently of an EC2 instance.

The experiment covers:

1. Creation of a Windows EC2 instance.
2. Creation of four IAM users with different access levels.
3. Organization of the users into Freshers and Seniors groups.
4. Assignment of EC2 read-only permissions to both groups.
5. Giving Seniors additional Start and Stop permissions.
6. Giving Senior-2 an additional Reboot permission.
7. Testing the different IAM permissions.
8. Creation of a separate EBS_Users group.
9. Giving Fresher-1 and Senior-1 permission to create, attach, and detach EBS volumes.
10. Creation of separate EBS volumes for a fresher and a senior.
11. Attaching the volumes to the Windows EC2 instance.
12. Configuring the volumes inside Windows as usable drives.
13. Creating test files on both EBS volumes.
14. Detaching the EBS volumes.
15. Terminating the original EC2 instance.
16. Launching a new Windows EC2 instance in the same Availability Zone.
17. Attaching the original EBS volumes to the new EC2 instance.
18. Verifying that the original files survived the termination of the original EC2 instance.

2. Experiment Overview

This experiment is divided into two connected parts.

The first part focuses on IAM-based access control. Four IAM users are created and organized into two groups: `Freshers` and `Seniors`. Both groups receive EC2 read-only access, while the Seniors receive additional permissions to start and stop the EC2 instance. Senior-2 also receives a separate permission to reboot the instance.

The second part focuses on EBS storage persistence. Fresher-1 and Senior-1 are added to the `EBS_Users` group and are given the permissions required to create, attach, and detach EBS volumes. The volumes are attached to the Windows EC2 instance, configured as Windows drives, and used to store simple test files.

The original EC2 instance is then terminated while the EBS volumes are kept separately. A new Windows EC2 instance is launched in the same Availability Zone, and the original EBS volumes are attached to it. The test files are then checked to confirm that the data remained available.

3. AWS Resources Used

The following resources were used during the experiment.

Resource	Value
Region	`ap-south-1`
Availability Zone	`ap-south-1b`
Operating System	Microsoft Windows Server 2022
Original EC2	`i-0a43e870c87d3fc54`
Replacement EC2	`i-0d3cc916a3e7135c9`
Original Instance Type	`t3.micro`

Table 1: AWS EC2 resources used in the experiment

3.1. IAM Users

- `Fresher-1`
- `Fresher-2`
- `Senior-1`
- `Senior-2`

3.2. IAM Groups

- `Freshers`
- `Seniors`
- `EBS_Users`

3.3. IAM Policies

- `AmazonEC2ReadOnlyAccess`
- `enior-Start-Stop-EC2`
- `Senior-2-Reboot`
- `EBS-Attach-Detach`

The custom Start/Stop policy is intentionally documented as `enior-Start-Stop-EC2`, which is the name actually used during the experiment.

3.4. EBS Volumes

- `Fresher-D-Volume`
- `Senior-F-Volume`

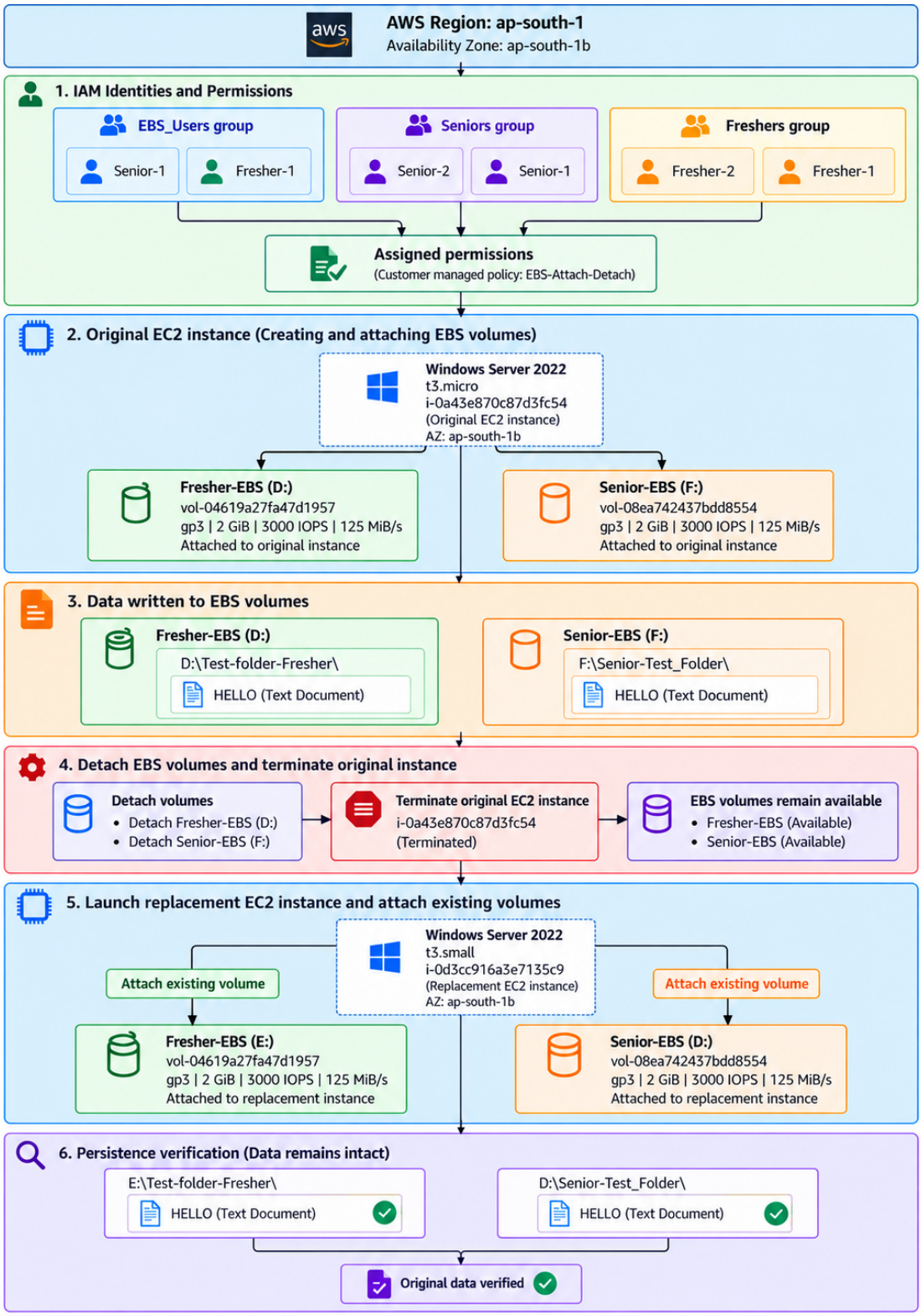
4. Topology

The topology shows the complete flow followed during this experiment. The experiment begins with the creation of a Windows EC2 instance and four IAM users. The users are divided into Freshers and Seniors so that different permission levels can be tested.

Both groups receive EC2 read-only access. The Seniors receive additional Start and Stop permissions through the `enior-Start-Stop-EC2` policy, while Senior-2 receives an additional Reboot permission through the `Senior-2-Reboot` policy.

For the storage portion, Fresher-1 and Senior-1 are added to the `EBS_Users` group. They create separate EBS volumes and attach them to the original Windows EC2 instance. Inside Windows, the attached volumes are configured as usable drives. Test files are then created on both volumes. The EBS volumes are detached before the original EC2 instance is terminated. A replacement Windows EC2 instance is then launched in the same Availability Zone. The existing EBS volumes are attached to the replacement instance and the original files are checked.

AWS IAM, EC2 and EBS Experiment Topology



Flow: IAM users and groups → permission assignment → original EC2 → EBS volumes → test data → detach and terminate EC2 → replacement EC2 → reattach EBS → verify persistent data.

5. Procedure

5.1. Step 1: Create the Windows EC2 Instance

Log in to the AWS Management Console and open the EC2 service.

From the left navigation panel, select **Instances** and click **Launch instances**.

Configure the instance with the following settings:

- Operating system: Microsoft Windows Server 2022
- Instance type: `t3.micro`
- Required key pair
- Required network and security configuration

Review the configuration and click **Launch instance**.

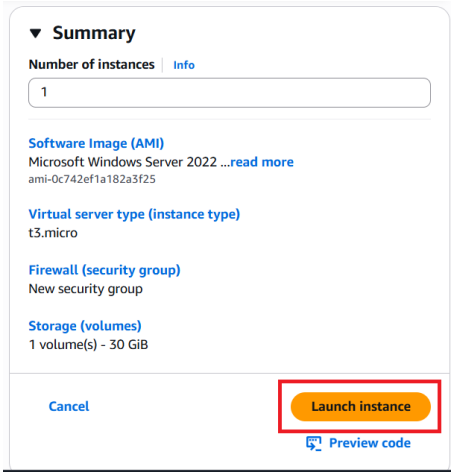


Figure 1: Windows EC2 instance launch configuration

After the launch is complete, open the instance list and verify that the instance is running in the `ap-south-1b` Availability Zone.

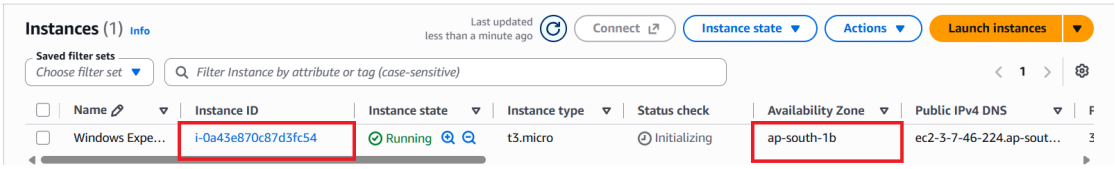


Figure 2: Running Windows EC2 instance in ap-south-1b

5.2. Step 2: Create the Four IAM Users

Open the AWS Management Console and go to:

IAM → Users → Create users

Create the following four users:

- `Fresher-1`
- `Fresher-2`
- `Senior-1`
- `Senior-2`

These users are created so that different access levels can be tested later in the experiment.

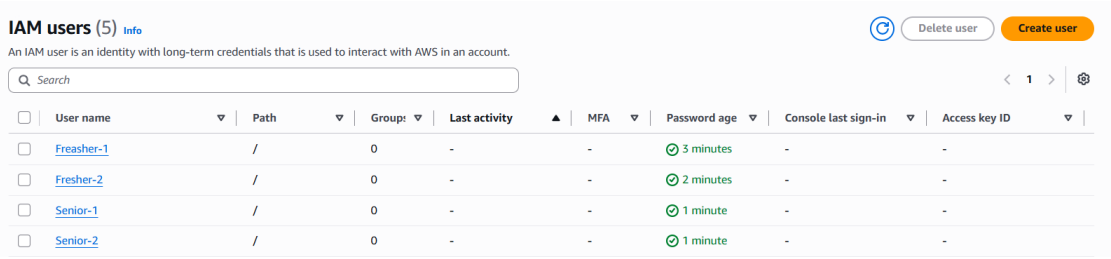


Figure 3: Creation of the four IAM users

5.3. Step 3: Create the Freshers Group

Go to:

IAM → User groups → Create group

Create a group named `Freshers`.
Add the following users to the group:

- `Fresher-1`
- `Fresher-2`

After creating the group, verify that both users appear as members.

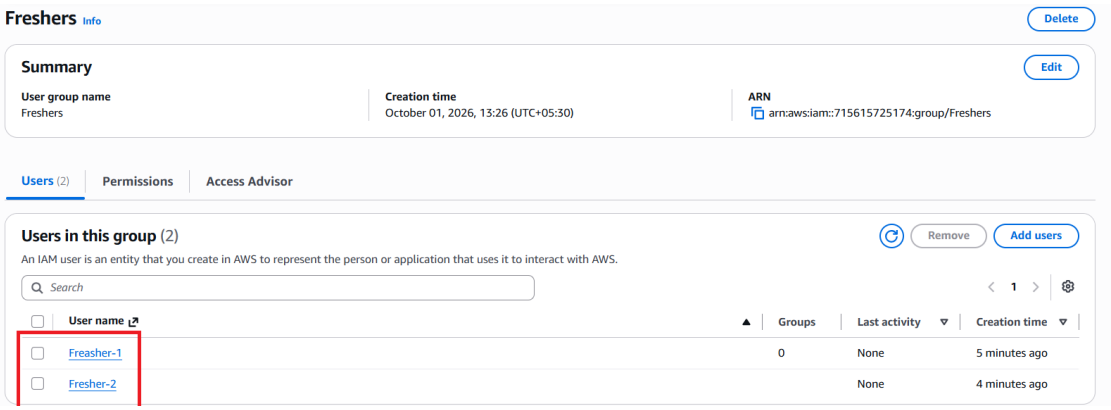


Figure 4: Freshers group with Fresher-1 and Fresher-2

5.4. Step 4: Create the Seniors Group

Create another IAM group named `Seniors`.
Add the following users:

- `Senior-1`
- `Senior-2`

Verify that both senior users are members of the group.

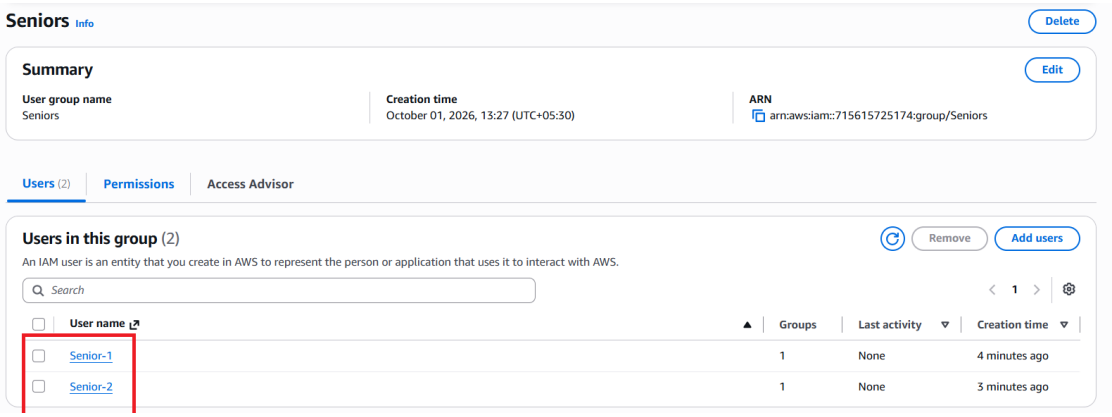


Figure 5: Seniors group with Senior-1 and Senior-2

5.5. Step 5: Give Freshers EC2 Read-Only Access

Open:

IAM → User groups → Freshers → Permissions

Click **Add permissions** and attach the AWS-managed policy:

`AmazonEC2ReadOnlyAccess`

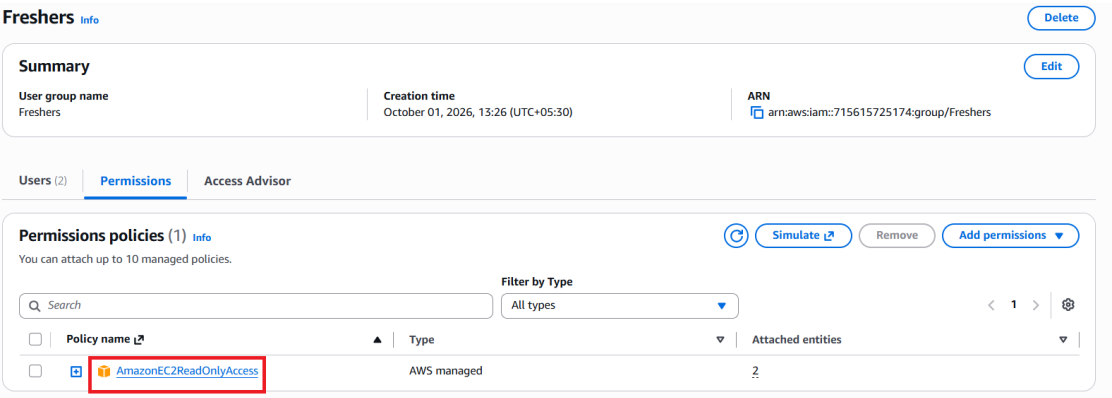


Figure 6: Attaching AmazonEC2ReadOnlyAccess to the Freshers group

This allows the Freshers to view EC2 information without giving them permission to start, stop, or reboot the instance.

5.6. Step 6: Give Seniors EC2 Read-Only Access

Open:

IAM → User groups → Seniors → Permissions

Click **Add permissions** and attach:

`AmazonEC2ReadOnlyAccess`

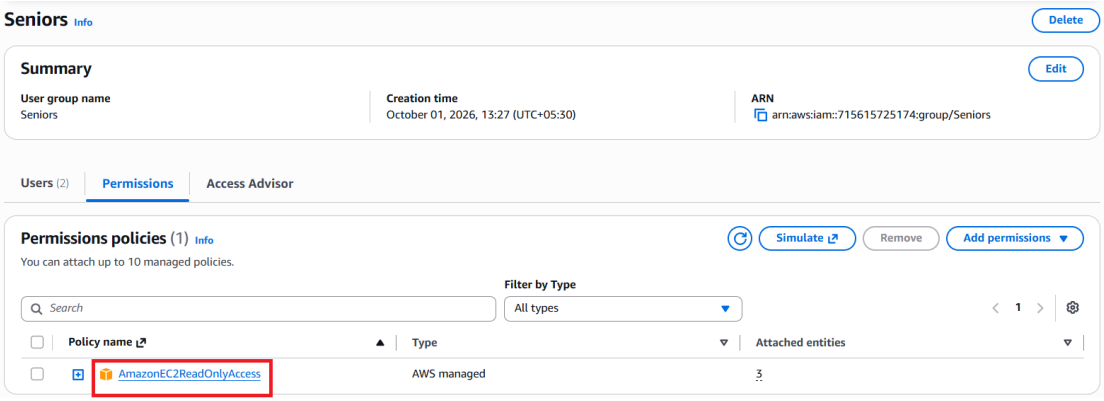


Figure 7: Attaching AmazonEC2ReadOnlyAccess to the Seniors group

The Seniors now have the same basic EC2 viewing permission as the Freshers. Additional permissions will be added separately.

5.7. Step 7: Create the Senior Start/Stop Policy

Go to:

IAM → Policies → Create policy

Create the custom policy with the exact name:

`enior-Start-Stop-EC2`

Allow the following EC2 actions:

- `ec2:StartInstances`
- `ec2:StopInstances`

Restrict the policy to the original Windows EC2 instance:

`i-0a43e870c87d3fc54`

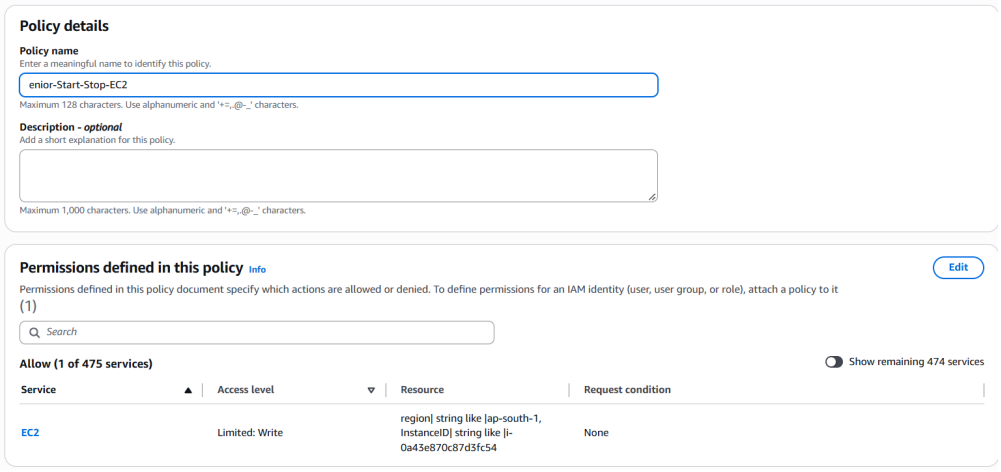


Figure 8: Creating the enior-Start-Stop-EC2 policy

Complete the policy creation and verify that it appears in the IAM Policies list.

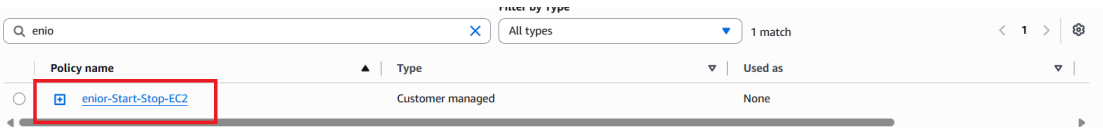


Figure 9: Created enior-Start-Stop-EC2 policy

This policy gives the Seniors Start and Stop control over the specific EC2 instance used in the experiment.

5.8. Step 8: Attach the Start/Stop Policy to Seniors

Go to:

IAM → User groups → Seniors → Permissions

Attach the custom policy:

`enior-Start-Stop-EC2`

The Seniors group should now have:

- `AmazonEC2ReadOnlyAccess`
- `enior-Start-Stop-EC2`

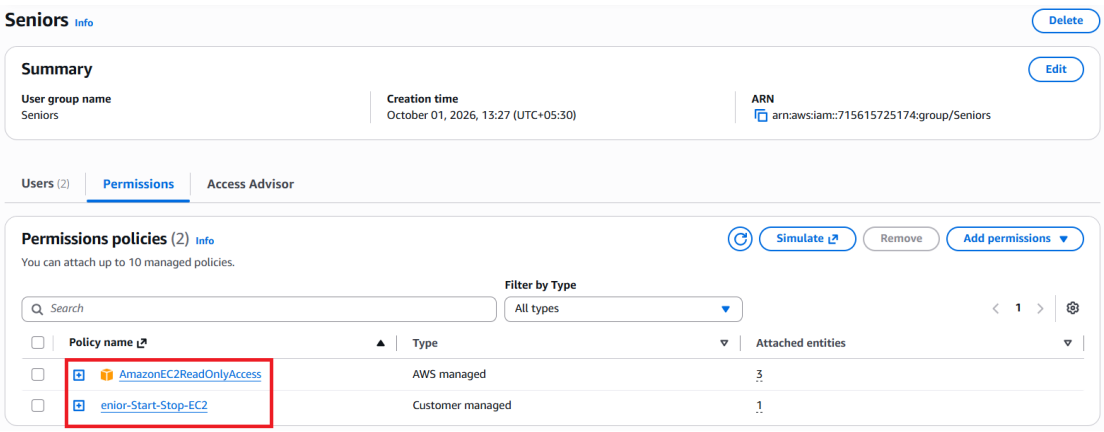


Figure 10: Seniors group with EC2 read-only and Start/Stop permissions

5.9. Step 9: Create Senior-2 Reboot Permission

Create an additional inline policy for `Senior-2`.

Use the policy name:

`Senior-2-Reboot`

Allow:

`ec2:RebootInstances`

Restrict the permission to:

`i-0a43e870c87d3fc54`

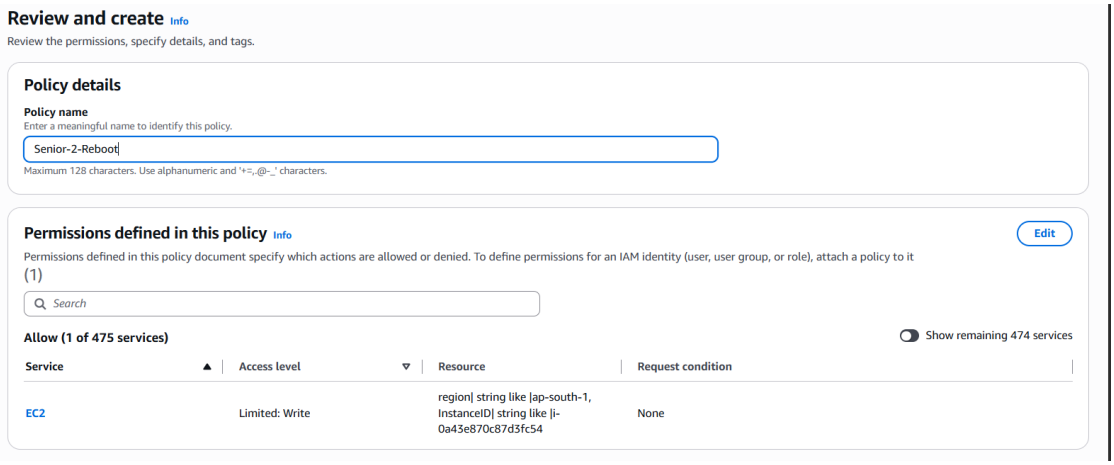


Figure 11: Creating the Senior-2-Reboot inline policy

After creating the policy, check Senior-2's permissions.

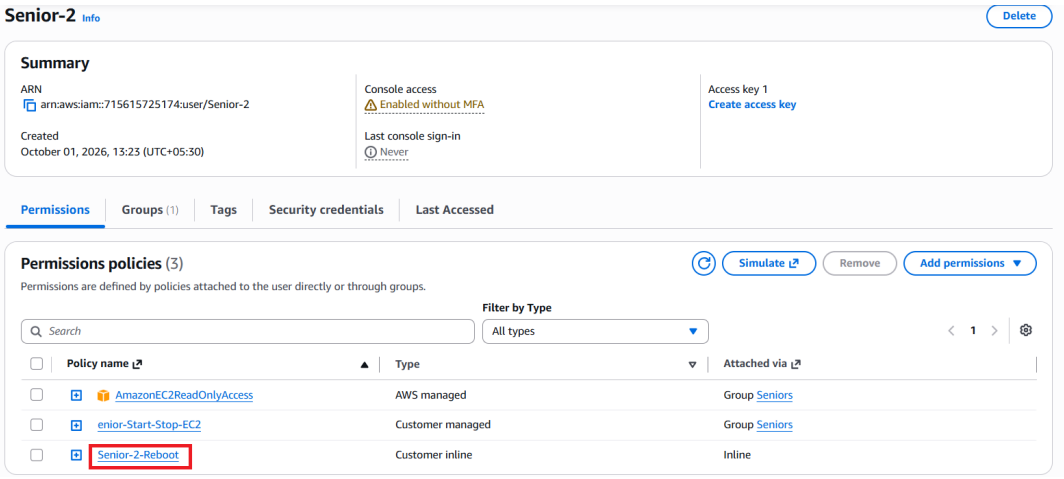


Figure 12: Senior-2 final permissions

Senior-2 therefore has an additional reboot permission that is not assigned to Senior-1.

6. Permission Testing

6.1. Step 10: Test Fresher Permissions

Sign in as `Fresher-1` and open:

EC2 → Instances

Verify that the Windows EC2 instance can be viewed.

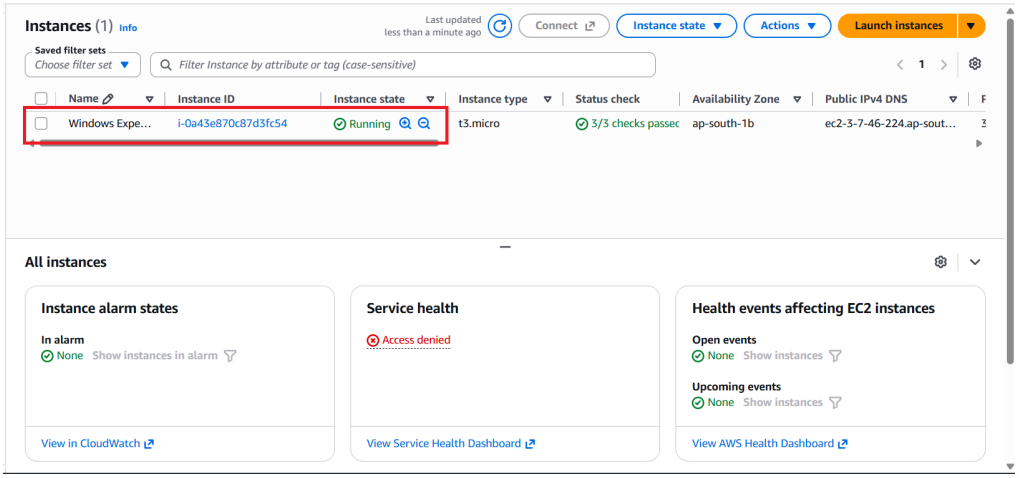


Figure 13: Fresher-1 viewing the EC2 instance

Now attempt to stop the EC2 instance.

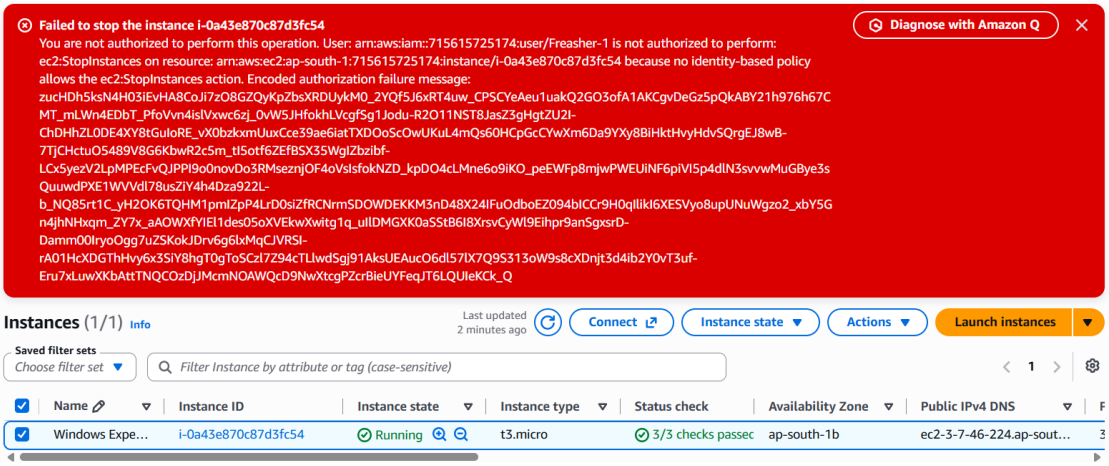


Figure 14: Fresher-1 denied permission to stop the EC2 instance

The instance can be viewed because of `AmazonEC2ReadOnlyAccess`, but the Stop operation is denied because Fresher-1 does not have `StopInstances` permission.

6.2. Step 11: Test Senior Start/Stop Permissions

Sign in as `Senior-1` and open the EC2 instance.

Attempt to stop the Windows EC2 instance.

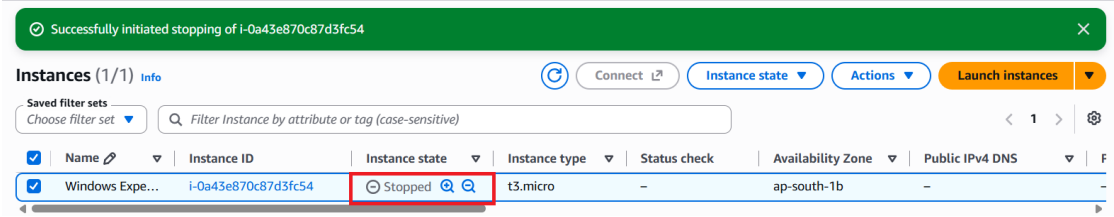


Figure 15: Senior-1 stopping the Windows EC2 instance

After the instance is stopped, start it again.

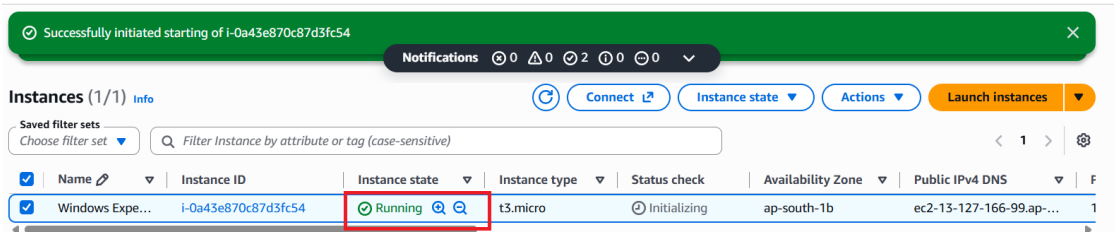


Figure 16: Senior-1 starting the Windows EC2 instance

The Start and Stop operations are available to Senior-1 because the Seniors group has the `enior-Start-Stop-EC2` policy.

6.3. Step 12: Test Senior-1 Reboot Restriction

Using `Senior-1`, attempt to reboot the EC2 instance.

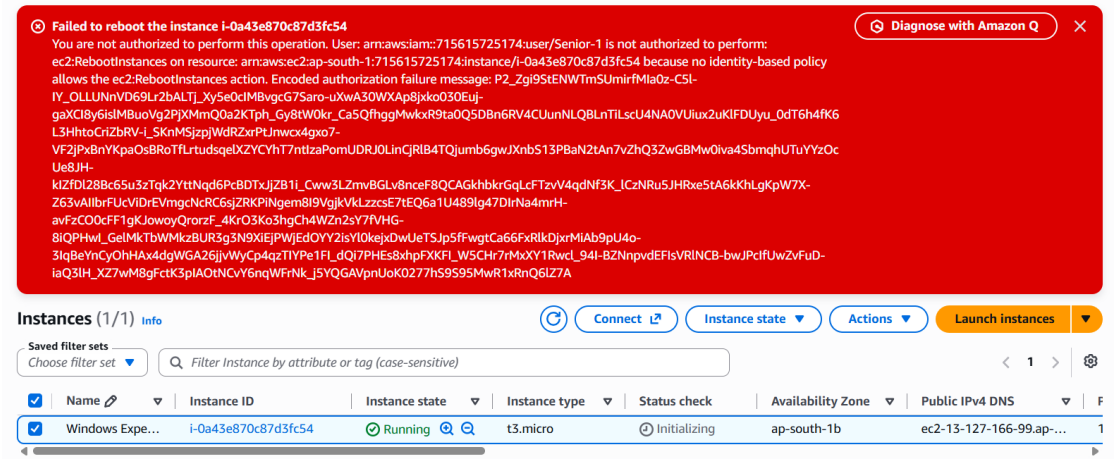


Figure 17: Senior-1 denied permission to reboot the EC2 instance

The reboot operation is denied because the reboot permission was assigned separately to Senior-2.

6.4. Step 13: Test Senior-2 Reboot Permission

Sign in as `Senior-2` and open the EC2 instance.

Select the reboot operation.

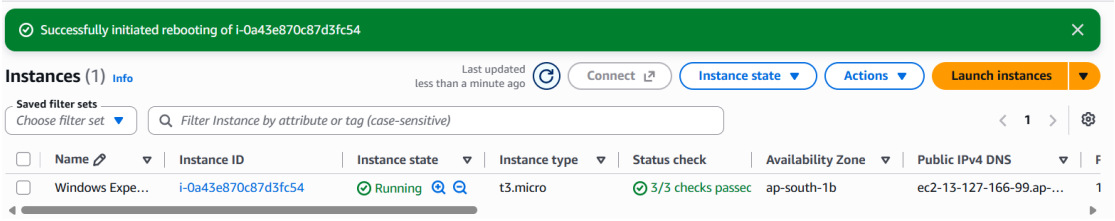


Figure 18: Senior-2 rebooting the EC2 instance

The operation succeeds because Senior-2 has the additional `Senior-2-Reboot` inline policy.

7. EBS Storage Configuration

7.1. Step 14: Create the EBS Users Group

Go to:

IAM → User groups → Create group

Create the group:

`EBS_Users`

Add:

- `Fresher-1`
- `Senior-1`

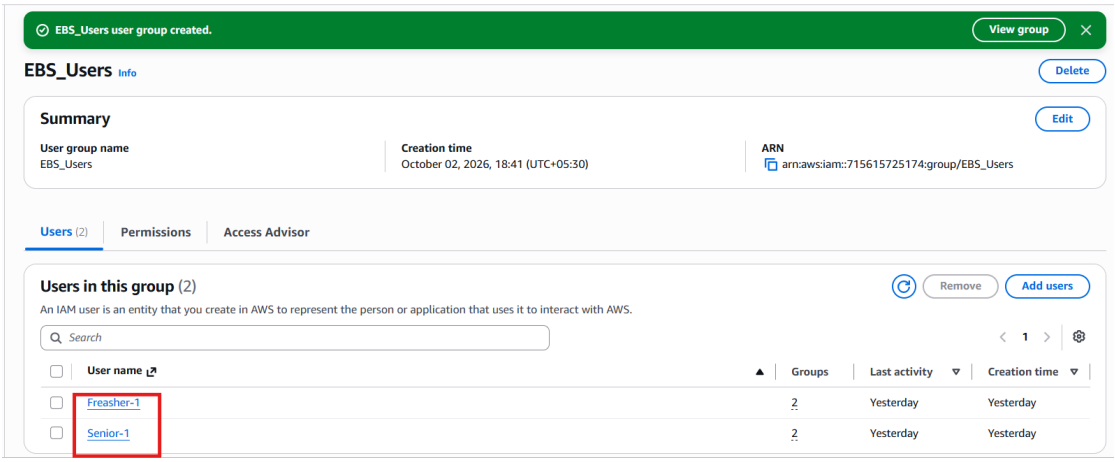


Figure 19: EBS_Users group with Fresher-1 and Senior-1

These two users will perform the EBS storage operations during the experiment.

7.2. Step 15: Create and Attach the EBS Permissions

Create the policy:

`EBS-Attach-Detach`

Give the group the required permissions for this experiment, including:

- `CreateVolume`
- `CreateTags`
- `AttachVolume`
- `DetachVolume`

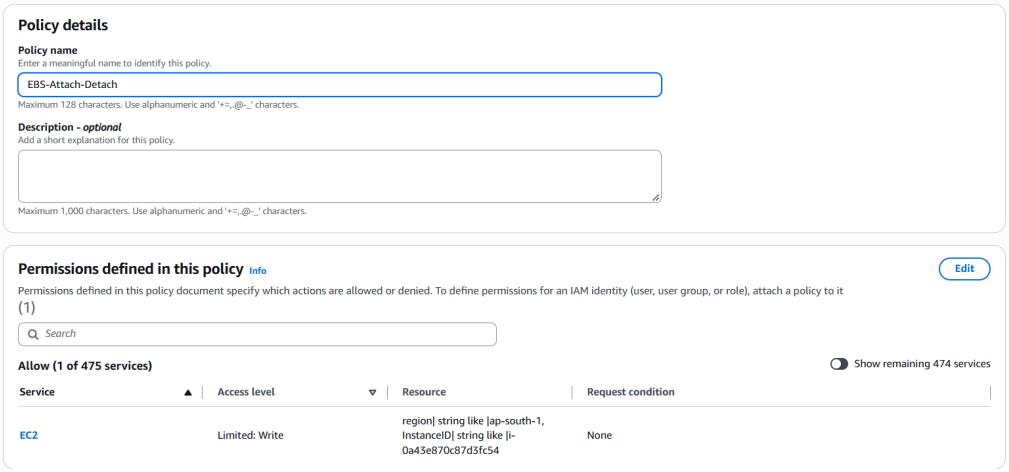


Figure 20: Creating the EBS-Attach-Detach policy

Attach the policy to the `EBS_Users` group.

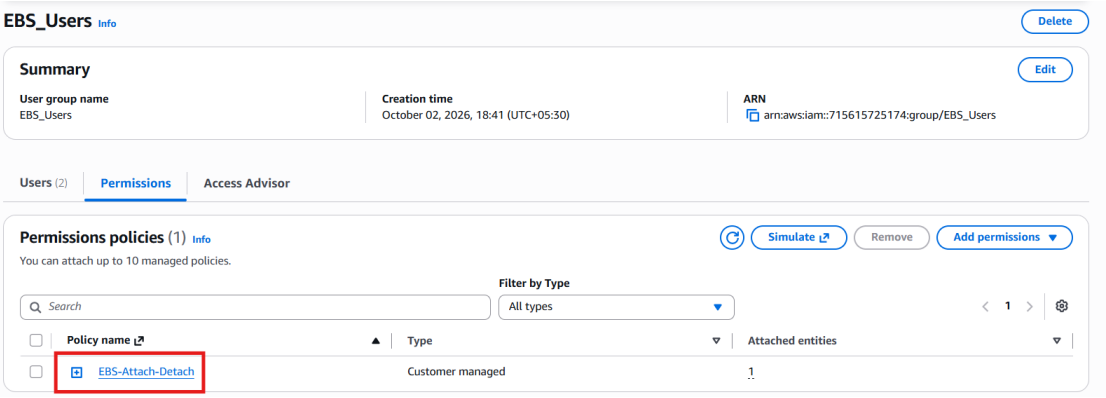


Figure 21: EBS-Attach-Detach policy attached to EBS_Users

The EBS operations in this experiment are performed through the assigned IAM users.

7.3. Step 16: Create Fresher's EBS Volume

Use `Fresher-1` for the EBS operation.

Go to:

EC2 → Volumes → Create volume

Configure the volume as follows:

- Volume type: `gp3`
- Size: `2 GiB`
- Availability Zone: `ap-south-1b`

Create the volume.

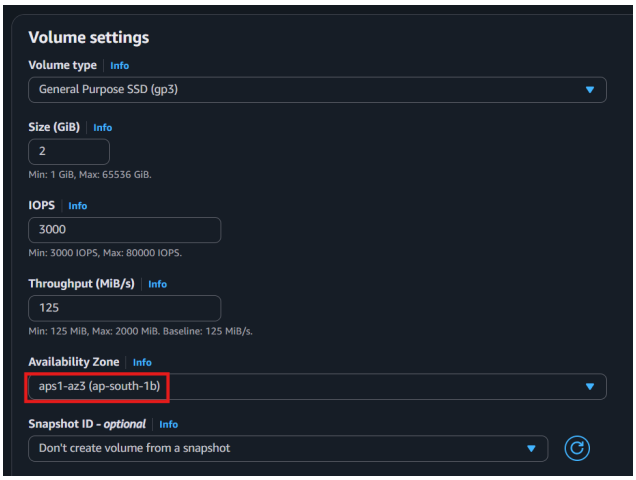


Figure 22: Creating Fresher's EBS volume

After creation, verify the volume in the Volumes list.

The volume is named:

`Fresher-D-Volume`

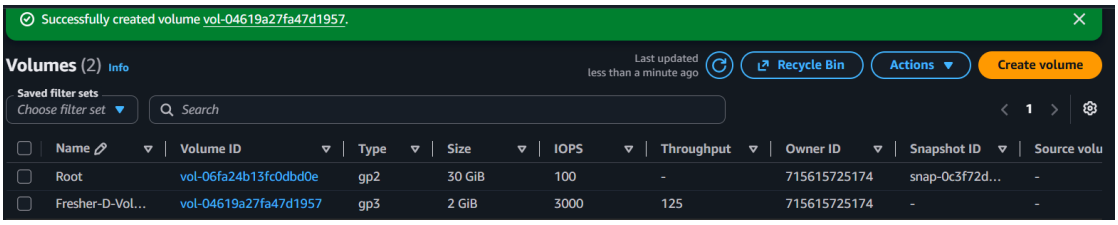


Figure 23: Fresher-D-Volume in the EBS Volumes list

The volume is created in the same Availability Zone as the EC2 instance so that it can be attached to that instance.

7.4. Step 17: Attach Fresher's EBS Volume

Select:

`Fresher-D-Volume`

Then select:

Actions → Attach volume

Select the original Windows EC2 instance:

`i-0a43e870c87d3fc54`

Attach the volume.

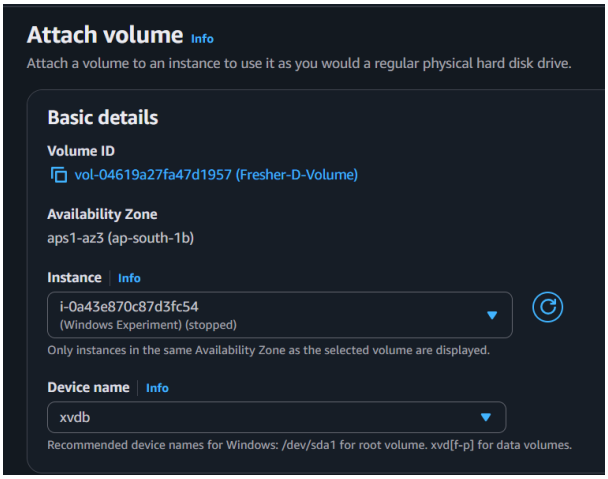


Figure 24: Attaching Fresher-D-Volume to the original EC2 instance

After attachment, verify that the volume changes to the **In-use** state.

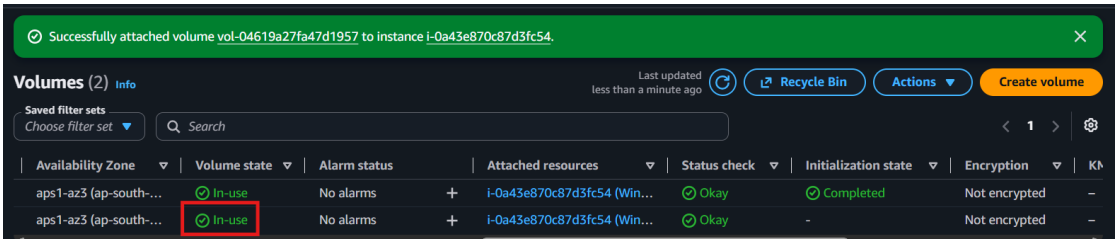


Figure 25: Fresher-D-Volume in the In-use state

7.5. Step 18: Create and Attach Senior's EBS Volume

Repeat the EBS volume creation process using `Senior-1`.
Create:

`Senior-F-Volume`

Use the following configuration:

- Volume type: `gp3`
- Size: `2 GiB`
- Availability Zone: `ap-south-1b`

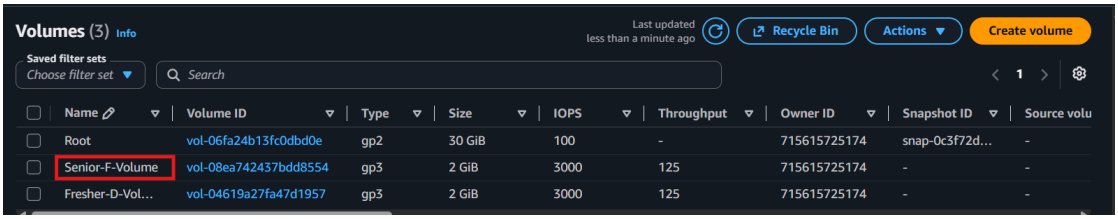


Figure 26: Creating Senior-F-Volume

Attach the volume to the same original Windows EC2 instance.
Verify that the volume is shown as **In-use**.

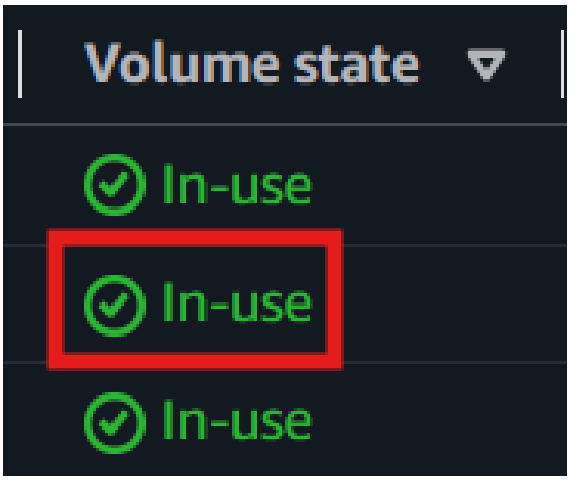


Figure 27: Senior-F-Volume attached to the original EC2 instance

7.6. Step 19: Configure the EBS Disks Inside Windows

Connect to the Windows EC2 instance using Remote Desktop.

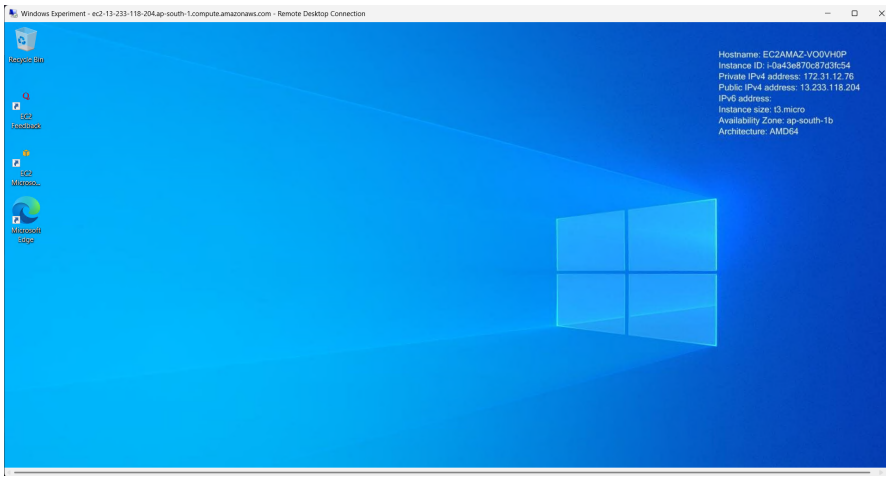


Figure 28: Remote Desktop connection to the Windows EC2 instance

Open the Windows disk management interface.

The newly attached EBS volumes appear as additional disks.

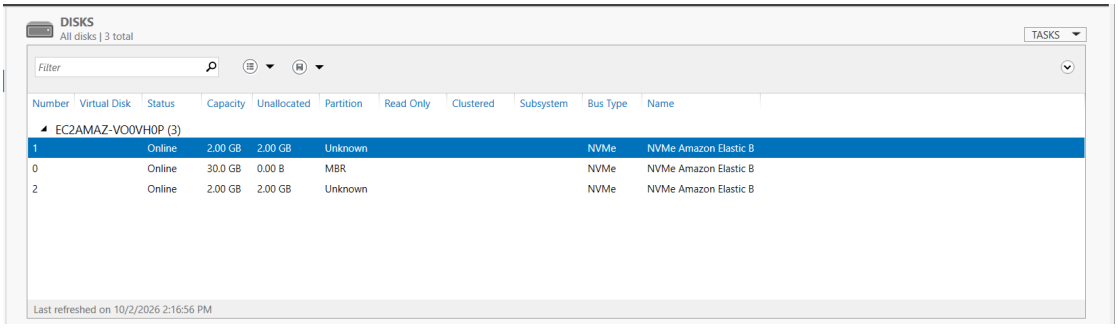


Figure 29: New EBS disks detected inside Windows

The disks are detected by Windows, but they still need to be configured with usable volumes and drive letters before they can be used normally from File Explorer.

7.7. Step 20: Configure Fresher EBS as a Windows Drive

Use the Windows New Volume Wizard and select the disk belonging to the Fresher EBS volume.

Create a new simple volume using:

- Drive letter: `D:`
- Volume label: `Fresher-EBS`
- File system: `NTFS`

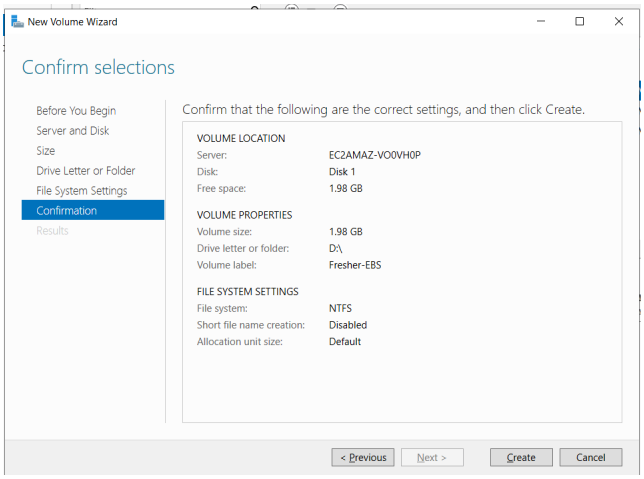


Figure 30: Configuring Fresher-EBS as a Windows volume

After the configuration is complete, verify that `Fresher-EBS` appears as drive `D:`.

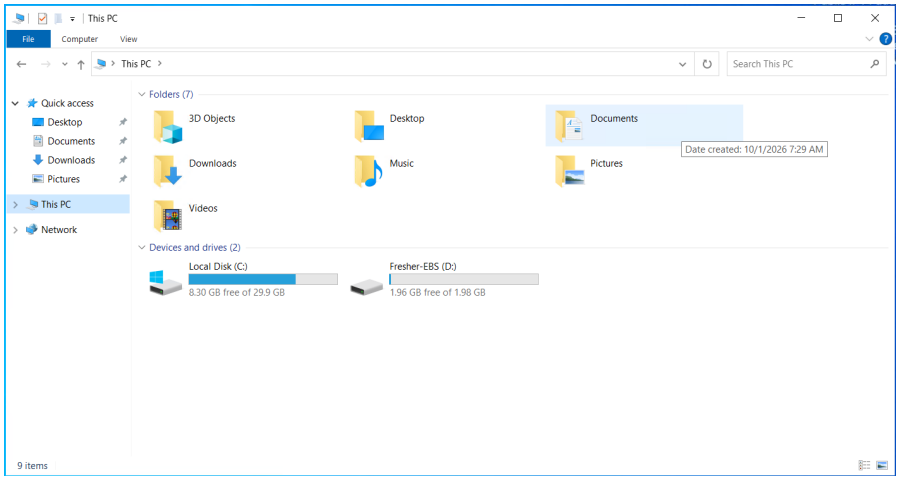


Figure 31: Fresher-EBS available as drive D:

7.8. Step 21: Configure Senior EBS as a Windows Drive

Use the same Windows volume configuration process for the Senior EBS disk.

Configure it using:

- Drive letter: `F:`
- Volume label: `Senior-EBS`
- File system: `NTFS`

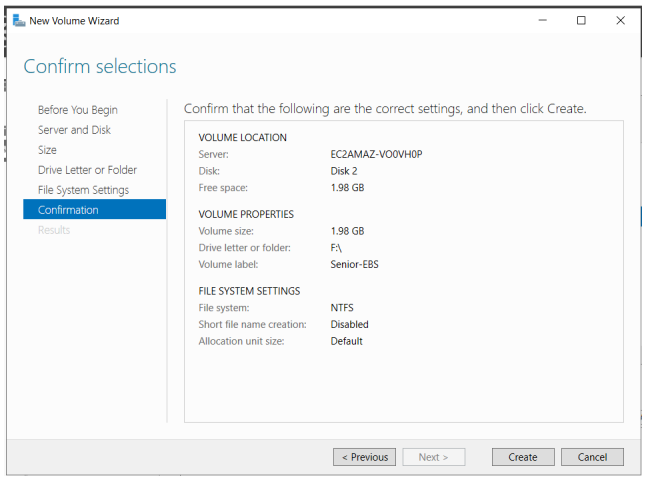


Figure 32: Configuring Senior-EBS as a Windows volume

After the configuration, verify that Windows shows:

- `C:`
- `Fresher-EBS (D:)`
- `Senior-EBS (F:)`

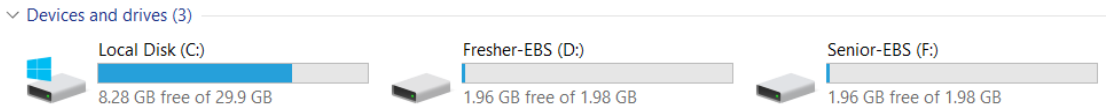


Figure 33: Fresher-EBS and Senior-EBS configured as Windows drives

8. EBS Persistence Test

8.1. Step 22: Create Test Data on Both EBS Volumes

Create a folder and an empty `HELLO` text file on the Fresher EBS volume.
Use the following location:

`D:\Test-folder-Fresher\HELLO`

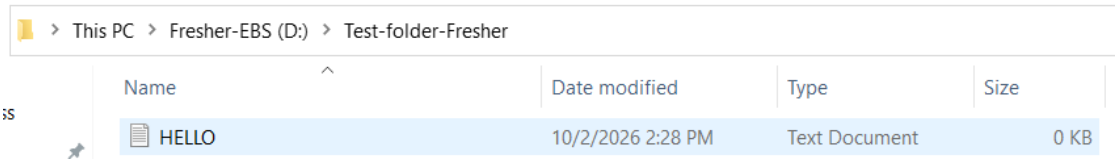


Figure 34: Test file created on the Fresher EBS volume

Next, create a folder and an empty `HELLO` text file on the Senior EBS volume.
Use:

`F:\Senior-Test_Folder\HELLO`

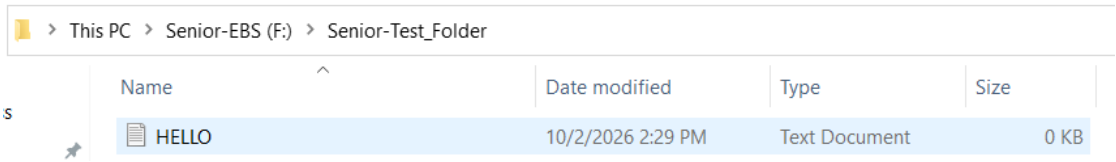


Figure 35: Test file created on the Senior EBS volume

These simple files are used as test data so that their presence can be checked after the EBS volumes are moved to another EC2 instance.

8.2. Step 23: Detach the EBS Volumes and Terminate the Original EC2

Go to:

EC2 → Volumes

Detach both EBS volumes from the original EC2 instance.
Verify that the volumes change to the **Available** state.

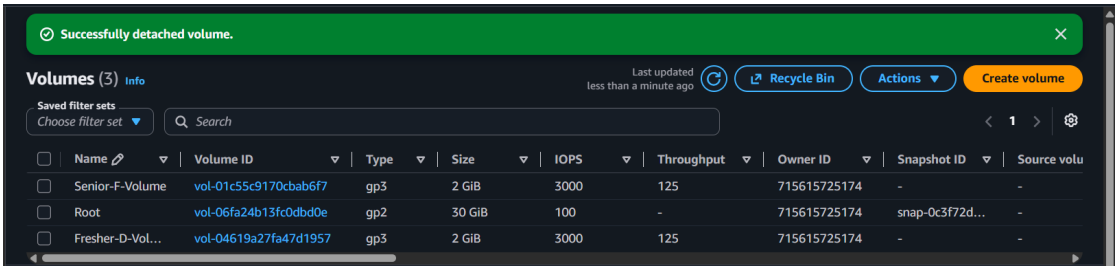


Figure 36: EBS volumes after being detached

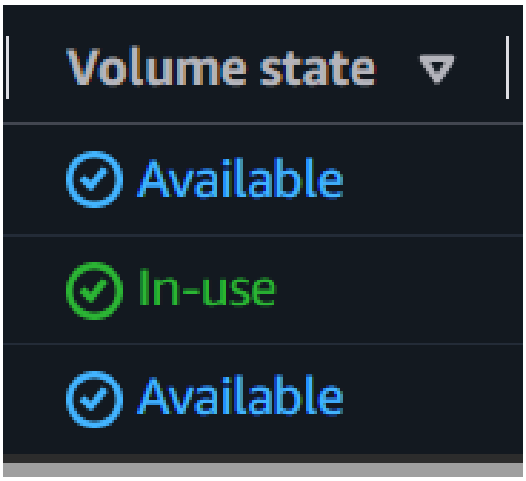


Figure 37: EBS volumes available for reattachment

Next, go to:

EC2 → Instances

Select the original instance:

`i-0a43e870c87d3fc54`

Select:

Instance state → Terminate instance

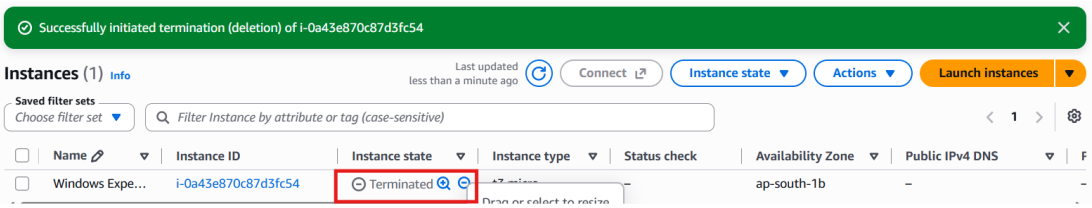


Figure 38: Terminating the original EC2 instance

The original EC2 instance is removed while the EBS volumes are kept separately.

8.3. Step 24: Create a New EC2, Reattach the EBS Volumes, and Verify the Data

Launch a replacement Windows EC2 instance in the same Availability Zone:

`ap-south-1b`

The replacement instance used in the experiment is:

`i-0d3cc916a3e7135c9`

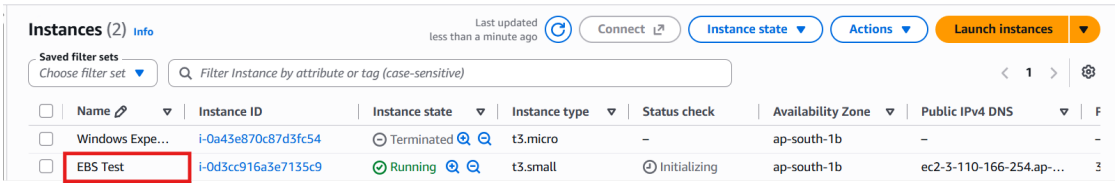


Figure 39: Replacement Windows EC2 instance

Attach the existing EBS volumes to the new EC2 instance and reconnect to the Windows machine.

After attaching the volumes, verify that Windows detects them.

The final drive-letter assignment observed in the experiment was:

- `Senior-EBS` → `D:`
- `Fresher-EBS` → `E:`

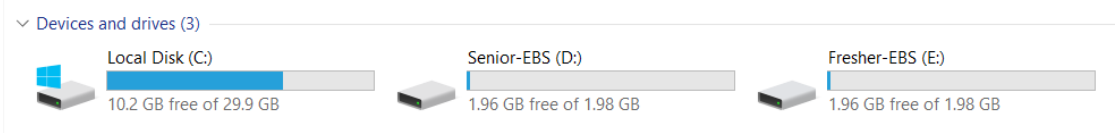


Figure 40: Existing EBS volumes detected on the replacement EC2

The drive letters changed after reattachment, but the EBS volumes and their data remained available.

Open the Senior volume and verify:

`Senior-Test_Folder` → `HELLO`

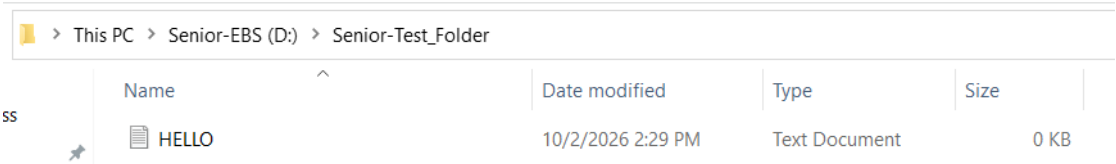


Figure 41: Senior EBS test file recovered on the replacement EC2

Open the Fresher volume and verify:

`Test-folder-Fresher` → `HELLO`

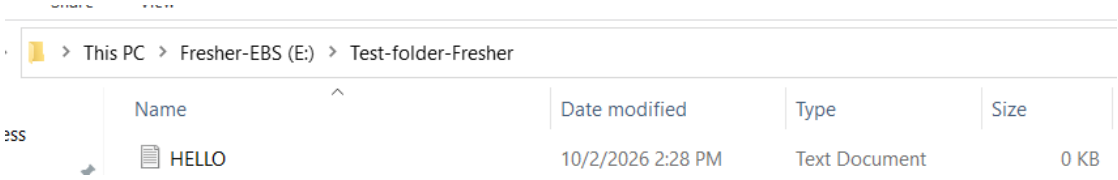


Figure 42: Fresher EBS test file recovered on the replacement EC2

The original EC2 instance was terminated, but both EBS volumes survived and the original test files were still present after the volumes were attached to the replacement EC2 instance.

9. Result / Observation

The experiment successfully demonstrated two main concepts.

9.1. IAM Access Control

Different users received different EC2 permissions according to their group membership and individual policies.

The Freshers could view EC2 resources but could not perform Start, Stop, or Reboot operations.

The Seniors received additional Start and Stop permissions.

Senior-2 received an additional Reboot permission that was not available to Senior-1.

The permission tests showed how IAM policies can provide different levels of access to different users.

9.2. EBS Persistence

The EBS volumes existed independently from the original EC2 instance.

After the original EC2 instance was terminated, the same EBS volumes were attached to the replacement Windows EC2 instance.

The final drive letters were:

- `Senior-EBS` → `D:`
- `Fresher-EBS` → `E:`

The drive letters changed after reattachment, but the data remained intact.

The original `HELLO` files were successfully found on both volumes after they were attached to the replacement EC2 instance.

10. Conclusion

The experiment demonstrated how AWS IAM can be used to provide different levels of access to EC2 resources through users, groups, and policies. The permission tests showed the difference between read-only access, Start/Stop access, and the additional Reboot access given to Senior-2.

The experiment also demonstrated EBS storage persistence. EBS volumes were created, attached to a Windows EC2 instance, configured as usable drives, and used to store test files. The original EC2 instance was then terminated, while the EBS volumes were preserved and attached to a replacement EC2 instance. The original files were still available, confirming that the data survived the replacement of the EC2 instance.